

Content-Oblivious Leader Election in 2-Edge-Connected Networks

Yi-Jun Chang^{*} Lyuting Chen[†] Haoran Zhou[‡]

Abstract

Censor-Hillel, Cohen, Gelles, and Sela (PODC 2022 & Distributed Computing 2023) studied *fully-defective* asynchronous networks, where communication channels may suffer an extreme form of alteration errors, rendering messages completely corrupted. The model is equivalent to *content-oblivious* computation, where nodes communicate solely via pulses. They showed that if the network is 2-edge-connected, then any algorithm for a noiseless setting can be simulated in the fully-defective setting; otherwise, no non-trivial computation is possible in the fully-defective setting. However, their simulation requires a predesignated leader, which they conjectured to be necessary for any non-trivial content-oblivious task.

Recently, Frei, Gelles, Ghazy, and Nolin (DISC 2024) refuted this conjecture for the special case of oriented ring topology. They designed two asynchronous content-oblivious leader election algorithms with message complexity $O(n \cdot \text{ID}_{\max})$, where n is the number of nodes and $\text{ID}_{\max}$ is the maximum ID. The first algorithm stabilizes in unoriented rings without termination detection. The second algorithm quiescently terminates in oriented rings, thus enabling the execution of the simulation algorithm after leader election.

In this work, we present an asynchronous content-oblivious leader election algorithm that quiescently terminates in any 2-edge connected network with message complexity $O(m \cdot N \cdot \text{ID}_{\min})$, where m is the number of edges, N is a known upper bound on the number of nodes, and $\text{ID}_{\min}$ is the smallest ID. Combined with the previous simulation result, our finding implies that any algorithm from the noiseless setting can be simulated in the fully-defective setting without assuming a preselected leader, entirely refuting the original conjecture.

^{*}National University of Singapore. ORCID: 0000-0002-0109-2432. Email: cyijun@nus.edu.sg

[†]National University of Singapore. ORCID: 0009-0002-8836-6607. Email: e0726582@u.nus.edu

[‡]National University of Singapore. ORCID: 0009-0001-2458-5344. Email: haoranz@u.nus.edu

Contents

1	Introduction	1
1.1	Our Contribution	3
1.2	Additional Related Work	4
2	Preliminaries	4
2.1	Content-Oblivious Model	4
2.2	Leader Election	5
2.3	DFS and Strongly Connected Orientation	6
3	Technical Overview	7
3.1	Synchronized Counting	7
3.2	DFS Notification	8
4	Quiescently Terminating Leader Election	10
4.1	Synchronized Counting	12
4.2	DFS Notification	15
5	Conclusions and Open Questions	23

1 Introduction

Fault tolerance is a cornerstone of distributed computing, enabling systems to remain operational despite various failures, such as node crashes or channel noise [BDM93; Dub13; KK20; Ray18]. One important category of faults, known as *alteration errors*, changes the content of messages but does not create new messages or eliminate existing ones. Traditional methods for handling such errors often rely on adding redundancy through coding techniques and assume known bounds on the frequency or severity of faults. However, in many practical settings, such assumptions may not hold, limiting the effectiveness of these approaches.

Fully-defective networks. Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] introduced the model of *fully-defective* asynchronous networks, in which all links are subject to a severe form of alteration errors: The content of every message can be arbitrarily modified, although the adversary cannot insert new messages or remove existing messages. Since messages can no longer carry meaningful information, the model is equivalent to *content-oblivious* computation, where communication is reduced to sending and receiving *pulses*. Algorithms in this setting operate entirely based on the patterns of pulse arrivals.

An impossibility result. A natural approach to designing a content-oblivious algorithm is to use unary encoding, representing messages as sequences of pulses. However, in the *asynchronous* setting, where no upper bound on message delivery time is known, there is no reliable way to detect the end of a sequence. Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] established a strong *impossibility* result: Let $f(x, y)$ be any non-constant function, then any two-party deterministic asynchronous content-oblivious communication protocol, where one party holds x and the other holds y , must fail to compute f correctly.

The impossibility result implies that, in a certain sense, no non-trivial computation is possible in any network that is *not* 2-edge-connected—that is, any network containing an edge e whose removal disconnects the graph into two components. By assigning control of each component to a different party, the problem reduces to the two-party setting to which the impossibility result applies.

Algorithm simulation over 2-edge-connected networks. Interestingly, Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] showed that in 2-edge-connected networks, any algorithm designed for the noiseless setting can be simulated in a fully defective network. We briefly sketch the main ideas underlying their approach below.

For the special case of an oriented ring, the challenge of termination detection can be addressed. Suppose a node wishes to broadcast a message. It can send a sequence of pulses that represents the unary encoding of the message in one direction and use the opposite direction to signal termination. To allow all nodes the opportunity to communicate, a token can be circulated in the ring, enabling each node to speak in turn.

To extend this approach to general 2-edge-connected networks, they leverage a result by Robbins [Rob39], which states that any 2-edge-connected undirected graph can be oriented to form a strongly connected directed graph. This guarantees the existence of a *Robbins cycle*, which is an oriented ring that traverses all nodes, possibly revisiting some nodes multiple times. They designed a content-oblivious algorithm that uses a depth-first search (DFS) to construct such a cycle.

Preselected leader assumption. A key limitation of their result is that it inherently relies on a *preselected leader*. For instance, a leader is needed to generate a unique token in a ring or to select the root for the DFS. Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] conjectured that the preselected leader assumption is essential for any non-trivial content-oblivious computation, leaving the following question open.

Question 1. *In 2-edge-connected networks, is it possible to simulate any algorithm designed for a noiseless setting in the fully defective model without relying on a preselected leader?*

Content-oblivious leader election on rings. To answer the above question affirmatively, it suffices to design a content-oblivious leader election algorithm that possesses certain desirable properties, enabling it to be composed with other algorithms. Recently, Frei, Gelles, Ghazy, and Nolin [FGG⁺24] answered this question for the special case of *oriented rings*. They designed two asynchronous content-oblivious leader election algorithms, both with message complexity $O(n \cdot \text{ID}_{\max})$, where n is the number of nodes and $\text{ID}_{\max}$ is the maximum identifier. The first algorithm *stabilizes* in unoriented rings, without termination detection. The second algorithm *quiescently* terminates in oriented rings, ensuring that no further pulses are sent to a node v after the algorithm on v declares termination.

The quiescent termination property enables the execution of the general simulation algorithm discussed earlier, following their leader election algorithm, as each node can correctly identify which pulses belong to which algorithm. As a result, this finding refutes the conjecture of Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] for the special case of oriented ring topologies. Given the work by Frei, Gelles, Ghazy, and Nolin [FGG⁺24], the next question to address is whether content-oblivious leader election is feasible for general 2-edge-connected networks.

Question 2. *In 2-edge-connected networks, is it possible to design a quiescently terminating content-oblivious leader election algorithm?*

Existing approaches for content-oblivious leader election heavily rely on the ring structure, making use of its two orientations to handle two types of messages. We briefly outline the key ideas behind the algorithms of Frei, Gelles, Ghazy, and Nolin [FGG⁺24] as follows.

A central building block of their algorithms is a stabilizing leader election algorithm on oriented rings, which operates as follows. Each node generates a token, and all tokens travel in the same direction. Each node v maintains a counter to track the number of tokens it has passed. Once the counter reaches $\text{ID}(v)$, node v destroys one token and temporarily elects itself as leader. Any incoming token will cause it to relinquish leadership. All counters eventually reach $\text{ID}_{\max}$ and stabilize. The node r with $\text{ID}(r) = \text{ID}_{\max}$ becomes the unique leader. This algorithm is not terminating, as nodes cannot determine whether additional tokens will arrive.

To achieve a quiescently terminating algorithm on oriented rings, they run the procedure in both directions sequentially, using the moment when the two counter values match as the termination condition. A node v begins the second run when its counter value from the first run reaches $\text{ID}(v)$.

To achieve a stabilizing algorithm on unoriented rings, they run the procedure in both directions concurrently. The correctness follows from the observation that the same node is selected as the leader in both executions.

1.1 Our Contribution

We answer Question 1 affirmatively under the assumption that an upper bound $N \geq n$ on the number of nodes $n = |V|$ is known, by presenting a content-oblivious leader election algorithm that quiescently terminates on any 2-edge-connected network.

Theorem 1. *There is a quiescently terminating leader election algorithm with message complexity $O(m \cdot N \cdot \text{ID}_{\min})$ in any 2-edge-connected network $G = (V, E)$, where $m = |E|$ is the number of edges, $N \geq n$ is a known upper bound on the number of nodes $n = |V|$, and $\text{ID}_{\min}$ is the smallest ID. Moreover, the leader is the last node to terminate.*

Combined with the simulation result of Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23], our finding implies¹ that in any 2-edge-connected network, any algorithm designed for the noiseless setting can be simulated in the fully-defective setting, without assuming a preselected leader, answering Question 2 affirmatively, again under the assumption that an upper bound $N \geq n$ is known. This refutes the original conjecture posed by Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23].

Table 1: New and old results on content-oblivious leader election.

Topology	Guarantee	Messages	Condition	Reference
Unoriented rings	Stabilizing	$O(n \cdot \text{ID}_{\max})$	$\times$	[FGG ⁺ 24]
Oriented rings	Quiescently terminating	$O(n \cdot \text{ID}_{\max})$	$\times$	[FGG ⁺ 24]
Oriented rings	Stabilizing	$\Omega(n \cdot \log \frac{\text{ID}_{\max}}{n})$	$\times$	[FGG ⁺ 24]
2-edge-connected	Quiescently terminating	$O(m \cdot N \cdot \text{ID}_{\min})$	Known N	Theorem 1

See Table 1 for a comparison between Theorem 1 and the results from prior work [FGG⁺24]. Although our algorithm has a higher message complexity of $O(m \cdot N \cdot \text{ID}_{\min})$ compared to the previous $O(n \cdot \text{ID}_{\max})$ bound in terms of network size, it reduces the dependency on identifiers from $\text{ID}_{\max}$ to $\text{ID}_{\min}$.

At first glance, our upper bound of $O(m \cdot N \cdot \text{ID}_{\min})$ may seem to contradict the lower bound of $\Omega(n \cdot \log \frac{\text{ID}_{\max}}{n})$ established in previous work [FGG⁺24]. However, the two results are consistent, as the lower bound does not make any assumptions about the value of $\text{ID}_{\min}$. Precisely, their bound is of the form $\Omega(n \cdot \log \frac{k}{n})$, where k is the number of distinct, *assignable* identifiers in the network. In other words, k is the size of the ID space.

While our algorithm offers improvements, it does require a mild assumption: All nodes must *a priori* agree on a known upper bound N on the number of nodes $n = |V|$ in the network, making it *non-uniform*. This assumption is widely adopted in many existing leader election and distributed graph algorithms, as upper bounds on network size are often available or can be estimated. In contrast, the algorithms from prior work [FGG⁺24] are *uniform*, requiring no prior knowledge of the network.

¹More precisely, as established in the prior work [FGG⁺24], a sufficient condition for achieving this objective consists of (1) quiescent termination and (2) the requirement that the leader is the final node to terminate. This is because the algorithmic simulation in [CCG⁺23] is initiated by the leader.

1.2 Additional Related Work

Fault-tolerant distributed computing involves two main challenges. The first is message corruption, which can occur due to factors such as channel noise. The second is the presence of faulty nodes and edges, which can arise from issues like unreliable hardware or malicious attacks.

To address message corruption, a common approach is to introduce additional redundancy using coding techniques, known as *interactive coding*. The noise affecting the messages can be either random or adversarial, and there is usually an upper bound on the level of randomness or adversarial behavior. The study of interactive coding was initiated by Schulman [Sch92; Sch93; Sch96] in the two-party setting and later extended to the multi-party case by Rajagopalan and Schulman [RS94]. For a comprehensive overview of interactive coding, see the survey by Gelles [Gel17].

For networks with n nodes, the maximum fraction of corrupted messages that a distributed protocol can tolerate is $\Theta(1/n)$ [JKL15]. If more than this fraction of messages are corrupted, the adversary can fully disrupt the communication of the node that communicates the fewest messages. Censor-Hillel, Gelles, and Haeupler [CGH19] presented a distributed interactive coding scheme that simulates any asynchronous distributed protocol while tolerating an optimal corruption of $\Theta(1/n)$ of all messages. A key technique underlying their algorithm is a content-oblivious BFS algorithm.

The celebrated impossibility theorem of Fischer, Lynch, and Paterson [FLP85] states that achieving consensus in an asynchronous distributed system is impossible for a deterministic algorithm when one or more nodes may crash. If the number of Byzantine edges is bounded by f , reliable communication can only be achieved if the graph is at least $2f$ -edge-connected [Dol82; Pel92].

A series of recent studies has focused on resilient and secure distributed graph algorithms in the synchronous setting [PY19b; PY19a; HP21b; HP21a; HPY22; Par22; FP23; HPY23]. These works have developed compilation schemes that transform standard distributed algorithms into resilient and secure versions. For instance, in a $(2f+1)$ -edge-connected network, any distributed algorithm in the **CONGEST** model can be adapted to remain resilient against up to f adversarial edges [HP21b].

Content-oblivious computation has also been studied in the synchronous setting. In the *beeping model* introduced by Cornejo and Kuhn [CK10], during each communication round, a node can either beep or listen. If the node listens, it can distinguish between silence or the presence of at least one beep. Several leader election algorithms have been developed for the beeping model and its variations [CD19; DBB18; FSW14; GH13; VZ25].

Several additional distributed models have been designed with limited communication capabilities to capture various real-world constraints. These include radio networks [CK03], population protocols [AAD⁺06], and stone-age distributed computing [EW13]. Leader election has been extensively studied in these models [ARV22; BGK20; CD19; CD21; CKP⁺19; DS18; EK21; GH13; SM20; VZ25].

2 Preliminaries

We present the model and problem considered in the paper and overview some basic terminology.

2.1 Content-Oblivious Model

The communication network is abstracted as a graph $G = (V, E)$, where each node $v \in V$ is a computing device and each edge $e \in E$ is a bidirectional communication link. We write $n = |V|$ and $m = |E|$. Each node v has a unique identifier $\text{ID}(v)$, and we define $\text{ID}_{\max} = \max_{v \in V} \text{ID}(v)$ and

$ID_{\min} = \min_{v \in V} ID(v)$. Throughout the paper, we assume that G is 2-edge-connected, meaning that the removal of any single edge does not disconnect the graph. We assume no prior knowledge of the network topology, except that each node is given an upper bound $N \geq n$ on the total number of nodes.

The degree of a node v , denoted by $\deg(v)$, is the number of edges incident to v . Although a node does not initially know the identifiers of its neighbors, it can distinguish among its incident edges using *port numbering*—a bijection between the set of incident edges and the set $\{1, 2, \dots, \deg(v)\}$.

In the content-oblivious setting, nodes communicate by sending *pulses*, which are messages without any content. We consider the *asynchronous* model, where node behavior is *event-driven*: A node can act only upon initialization or upon receiving a pulse. Based on the pattern of previously received pulses, each node decides its actions, which may include sending any number of pulses through any subset of its ports. We emphasize that when a node receives a pulse, it knows the port through which the pulse arrived. Likewise, a node can specify how many pulses to send through each port. We restrict our attention to the *deterministic* setting, in which nodes do not use randomness in their decision-making.

Time. There is no upper bound on how long a pulse may take to traverse an edge, but every pulse is guaranteed to be delivered after a finite delay. An equivalent way to model this behavior is by assuming the presence of a scheduler. At each time step, as long as there are pulses in transit, the scheduler arbitrarily selects one and delivers it.

We assume that upon receiving a pulse, all actions triggered by that pulse are performed *instantaneously*, so that the system can be analyzed in *discrete* time steps. We focus only on specific, well-defined moments in the execution of the algorithm—namely, the moments just before the scheduler selects a pulse to deliver. At these times, all the actions resulting from earlier pulse deliveries have been completed, and the system is in a stable state, awaiting the next step. We do not analyze the system during any intermediate period in which a node is still performing the actions triggered by a pulse arrival.

Counting the number of pulses. Throughout the paper, we write $[s] = \{1, 2, \dots, s\}$. For each node $v \in V$ and each port number $i \in [\deg(v)]$, we use the following notation to track the number of pulses sent and received so far:

- $\sigma_i(v)$ denotes the number of pulses sent on port i of node v .
- $\rho_i(v)$ denotes the number of pulses received on port i of node v .

2.2 Leader Election

We now define the *leader election* problem.

Definition 2 (Leader election). *The task of leader election requires that each node outputs exactly one of Leader and Non-Leader, with the additional requirement that exactly one node outputs Leader.*

More precisely, each node maintains a binary internal variable `isLeader` to represent its current output, taking the value `Leader` or `Non-Leader`. The value of `isLeader` may change arbitrarily many times during the execution of the algorithm, but it must eventually stabilize to a final value that

remains unchanged thereafter. The final outputs of all nodes must collectively satisfy the conditions of Definition 2.

Due to the asynchronous nature of our model, there are various ways in which an algorithm can *finish*. Based on this, we classify algorithms into three types, ordered by increasing strength of termination guarantees.

Definition 3 (Stabilization). *A node stabilizes at time t if its output does not change after time t . An algorithm is stabilizing if it guarantees that all nodes eventually stabilize.*

Definition 4 (Termination). *A node terminates at time t if it explicitly declares termination at time t . After declaring termination, the node ignores all incoming pulses and can no longer change its output. An algorithm is terminating if it guarantees that all nodes eventually terminate.*

Definition 5 (Quiescent termination). *A node quiescently terminates at time t if it terminates at time t and no further pulses arrive at the node thereafter. An algorithm is quiescently terminating if it guarantees that all nodes eventually quiescently terminate.*

Ideally, we aim to design leader election algorithms that are quiescently terminating, as this property allows a second algorithm to be executed safely after leader election. In contrast, if the algorithm guarantees only stabilization or termination, nodes may be unable to distinguish whether an incoming pulse belongs to the leader election process or the subsequent algorithm. This ambiguity can affect the correctness of both algorithms. For this reason, prior work [FGG⁺24] establishes a general algorithm simulation result only for oriented rings, but not for unoriented rings.

2.3 DFS and Strongly Connected Orientation

We describe a DFS tree T_G of the graph $G = (V, E)$ that is *uniquely determined* by the port numbering and node identifiers. Based on this DFS tree, we describe an edge orientation of G that transforms it into a strongly connected directed graph $\vec{G}$. Both the DFS tree and the orientation play a critical role in the analysis of our algorithm.

DFS. Throughout this paper, we write r to denote the node with the smallest identifier. Consider a DFS traversal rooted at r such that when a node selects the next edge to explore among its unexplored incident edges, it chooses the one with the *smallest* port number. Let $T_G = (V, E_T)$ be the resulting DFS tree. The edges in E_T are called *tree edges*, and the remaining edges $E_B = E \setminus E_T$ are called *back edges*. The following observation is folklore.

Observation 6. *For any edge $\{u, v\} \in E_B$, either u is an ancestor of v , or v is an ancestor of u .*

Proof. When a node u explores an incident edge $e = \{u, v\}$ during the DFS, there are two possibilities: Either v is an ancestor of u , or v has not been visited yet. In the first case, $\{u, v\} \in E_B$. In the second case, v is discovered for the first time, so $\{u, v\} \in E_T$ is added as a tree edge in E_T . $\square$

Definition 7 (Strongly connected orientation). *The directed graph $\vec{G} = (V, \vec{E}_T \cup \vec{E}_B)$ is defined by*

$$\begin{aligned}\vec{E}_T &= \{(u, v) \mid \{u, v\} \in E_T \text{ and } u \text{ is the parent of } v \text{ in } T_G\}, \\ \vec{E}_B &= \{(u, v) \mid \{u, v\} \in E_B \text{ and } v \text{ is an ancestor of } u \text{ in } T_G\}.\end{aligned}$$

A directed graph is *strongly connected* if for any two vertices u and v in the graph, there is a directed path from u to v . We have the following observation.

Observation 8. *If G is 2-edge-connected, then $\vec{G}$ is strongly connected.*

Proof. To prove the observation, it suffices to show that, for any tree edge $e = \{u, v\}$, both u and v are in the same strongly connected component of $\vec{G}$, as this implies that all nodes in V are in the same strongly connected component. Let u be the parent of v . By Definition 7, we already know that (u, v) is a directed edge of $\vec{G}$, so it remains to show that u is reachable from v in $\vec{G}$.

Let S be the set consisting of v and all descendants of v in the DFS tree T_G . Since $G = (V, E)$ is 2-edge-connected, there exists an edge $e' \in E \setminus \{e\}$ that connects $V \setminus S$ and S . Since e is the only tree edge connecting $V \setminus S$ and S , e' must be a back edge. Let $e' = \{x, y\}$ where y is an ancestor of x in T_G .

Since $x \in S$, either $x = v$ or x is a descendant of v , so there is a directed path $P_{v \rightarrow x}$ from v to x using only edges in $\vec{E}_T$. Since $y \in V \setminus S$ is an ancestor of x , either $y = u$ or y is an ancestor of u , so there is a directed path $P_{y \rightarrow u}$ from y to u using only edges in $\vec{E}_T$. Combining $P_{v \rightarrow x}$, (x, y) , and $P_{y \rightarrow u}$, we infer that u is reachable from v in $\vec{G}$. $\square$

Shortest paths. Throughout the paper, for any two nodes $u \in V$ and $v \in V$, we write $P_{u,v}$ to denote any shortest path from u to v in $\vec{G}$, whose existence is guaranteed by Observation 8. We allow $u = v$, in which case $P_{u,v}$ is path of zero length consisting of a single node $u = v$.

3 Technical Overview

We present a technical overview for the proof of Theorem 1. Our algorithm aims to elect the node r with the smallest identifier as the leader. It operates in two phases. In the first phase, each node maintains a counter that is incremented in an almost synchronized manner using the α -synchronizer of Awerbuch [Awe85]. A node v declares itself the leader once its counter reaches $\text{ID}(v)$. This first phase alone yields a simple *stabilizing* leader election. To ensure *quiescent termination*, the second phase is executed, in which the elected leader announces its leadership through a DFS traversal.

3.1 Synchronized Counting

We begin by describing the first phase of the algorithm.

Counting. To achieve an almost synchronized counting, we employ the α -synchronizer of Awerbuch [Awe85], as follows. Each node maintains a counter variable $\text{Count}(v)$, which is initialized to 0 at the start of the algorithm. A node increments its counter by one after receiving a new pulse from each of its neighbors. Formally, the counter value of a node v is given by

$$\text{Count}(v) = \min_{j \in [\deg(v)]} \rho_j(v).$$

Each node broadcasts a pulse to all its neighbors upon initialization and immediately after each counter increment.

This process is *non-blocking*: In the absence of an exit condition, nodes will continue counting together indefinitely. Furthermore, no node can advance significantly faster than its neighbors: For

any pair of adjacent nodes, their counter values differ by at most one. A faster-counting node must wait for a pulse from a slower-counting neighbor before it can increment its counter again.

Stabilizing leader election. A node v elects itself as the leader once

$$\text{Count}(v) = \text{ID}(v).$$

After electing itself as the leader, node v *freezes* its counter at $\text{Count}(v) = \text{ID}(v)$ and does not increment it further. This action caps the counter value of any neighbor of v at most $\text{ID}(v) + 1$. More generally, in a graph of $n \leq N$ nodes, the counter value of any node is upper bounded by $\text{ID}(v) + N - 1$, since any two nodes are connected by a path of length at most $N - 1$.

To ensure that the algorithm elects the node r with the smallest identifier as the *unique* leader, it suffices for any two identifiers to differ by at least N . This can be achieved by first multiplying each identifier by N . This is where the *a priori* knowledge of N comes into play.

At this stage, we have a simple stabilizing leader election algorithm. However, it remains insufficient for our purposes: We also want all non-leader nodes to recognize their non-leader status and terminate in a quiescent manner.

3.2 DFS Notification

We proceed to describe the second phase of the algorithm. The leader must find a way to notify all other nodes to settle as non-leaders. This seems challenging because the leader can only send additional pulses, which appear indistinguishable from those in the first phase. As we will see, assuming the network is 2-edge-connected, there is a simple and effective approach: Just send many pulses to overwhelm the listener.

Notifying a neighbor. Let u be a neighbor of the leader r , and let the edge $e = \{u, r\}$ correspond to port i at r and port j at u . The leader r notifies u of its leadership as follows:

1. Wait until $\rho_i(r) \geq \text{ID}(r) + 1$.
2. Send pulses to u until $\sigma_i(r) = \text{ID}(r) + N + 2$.

Since the graph G is 2-edge-connected, there exists a path P from u to r that avoids e , implying

$$\text{Count}(u) \leq \text{Count}(r) + \text{length}(P) \leq \text{ID}(r) + N - 1.$$

Eventually, u receives $\text{ID}(r) + N + 2$ pulses from r , so there must be a moment where

$$\rho_j(u) \geq \text{Count}(u) + 3,$$

triggering anomaly detection. Under normal conditions in the first phase, u would expect

$$\rho_j(u) \leq \sigma_i(r) = \text{Count}(r) + 1 \leq \text{Count}(u) + 2,$$

so this deviation signals a violation.

Additionally, the waiting condition $\rho_i(r) \geq \text{ID}(r) + 1$ ensures that $\text{Count}(u) \geq \text{ID}(r)$. Thus, u not only recognizes that it is not the leader but can also deduce the exact value of $\text{ID}(r)$ using

$$\text{ID}(r) = N \cdot \left\lfloor \frac{\text{Count}(u)}{N} \right\rfloor,$$

since all node identifiers are integer multiples of N .

DFS. Intuitively, the leader r is able to notify a neighbor u because the counter value of node u is constrained by a *chain* P , whose other end is *anchored* at the leader r , whose counter has already been frozen. To extend this notification process to the entire network, we perform a DFS, using the same notification procedure to explore each new edge. When a node finishes exploring all its remaining incident edges, it can notify its parent by sending pulses until a total of $\text{ID}(r) + N + 2$ pulses have been sent through this port since the start of the algorithm. When a node u explores a back edge $\{u, v\}$, the node v replies using the same method. When a node decides which new incident edge to explore, it prioritizes the edge with the smallest port number, so the resulting DFS tree is T_G , as defined in Section 2.3.

Quiescent termination. To see that the algorithm achieves quiescent termination, observe that by the end of the execution, for each edge e , the number of pulses sent in both directions is exactly $\text{ID}(r) + N + 2$. Therefore, once a node observes that it has both sent and received exactly $\text{ID}(r) + N + 2$ pulses along each of its ports and has completed all local computations, it can safely terminate with a quiescent guarantee.

Correctness. A similar chain-and-anchor argument can be applied to analyze the correctness of the DFS-based notification algorithm. We prove by induction that for each node u that has been visited by DFS, its frozen counter value satisfies

$$\text{Count}(u) \leq \text{ID}(r) + \text{length}(P_{u,r}).$$

Recall from Section 2.3 that $P_{a,b}$ is a shortest path from a to b in the directed version $\vec{G}$ of G defined in Definition 7.

Suppose during the DFS, a node x is about to explore an incident edge $e = \{x, y\}$ using the notification procedure, and that y has not yet been visited. Let w be the first node on the path $P_{y,r}$ that is either x or an ancestor of x in the DFS tree T_G , then we have

$$\text{length}(P_{y,r}) = \text{length}(P_{y,w}) + \text{length}(P_{w,r}).$$

We now apply the chain-and-anchor argument using the chain $P_{y,w}$ and anchor w . This choice of chain is valid because the path $P_{y,w}$ cannot include the edge e , as e is oriented from x to y in $\vec{G}$. This is precisely why the analysis is carried out in $\vec{G}$ rather than in G .

By the induction hypothesis, the frozen counter value at the anchor satisfies $\text{Count}(w) \leq \text{ID}(r) + \text{length}(P_{w,r})$. It follows that

$$\begin{aligned} \text{Count}(y) &\leq \text{Count}(w) + \text{length}(P_{y,w}) \\ &\leq \text{ID}(r) + \text{length}(P_{w,r}) + \text{length}(P_{y,w}) \\ &= \text{ID}(r) + \text{length}(P_{y,r}) \\ &\leq \text{ID}(r) + N - 1. \end{aligned}$$

Therefore, the chain-and-anchor argument works, allowing y to be successfully notified. Moreover, the frozen counter value of y also satisfies the induction hypothesis.

In addition, we must ensure that the DFS procedure proceeds correctly and no event is triggered unexpectedly. Once again, the chain-and-anchor argument implies that the condition

$$\text{Count}(u) \leq \text{ID}(r) + \text{length}(P_{u,r}) \leq \text{ID}(r) + N - 1 < \text{ID}(u)$$

holds for all nodes $u \in V \setminus \{r\}$ at all times. This ensures that no node other than r can be mistakenly elected as a leader. It also guarantees that pulses sent by nodes still in the first phase cannot accidentally trigger a notification in the second phase. Specifically, if u is still in the first phase, then for every port $j \in [\deg(u)]$, we have

$$\sigma_j(u) = \text{Count}(u) + 1 \leq \text{ID}(r) + N,$$

which remains below the threshold of $\text{ID}(r) + N + 2$ pulses required to confirm receipt of a notification.

Remark. One might wonder why the notification must follow a DFS rather than allowing a node to notify all neighbors simultaneously to accelerate the process. A key reason is that doing so can break the chain-and-anchor argument by eliminating anchors. For instance, in a ring topology, if the node r with the smallest identifier simultaneously sends multiple pulses to *both* neighbors, all of the pulses might be interpreted as part of the first phase, with no detectable anomaly, regardless of how many pulses are sent.

Both our leader election algorithm and the simulation algorithm of Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23] utilize DFS, but for different purposes. They perform a DFS from a pre-selected leader to find a cycle, while we use DFS to disseminate the identity of the elected leader to all nodes and to achieve quiescent termination. Coincidentally, both approaches leverage the strongly connected orientation of 2-edge-connected graphs. In our case, the orientation $\vec{G}$ is used solely in the analysis to facilitate the chain-and-anchor argument. In their case, it guarantees the existence of a Robbins cycle, which plays a critical role in their algorithm.

4 Quiescently Terminating Leader Election

In this section, we prove our main theorem.

Theorem 1. *There is a quiescently terminating leader election algorithm with message complexity $O(m \cdot N \cdot \text{ID}_{\min})$ in any 2-edge-connected network $G = (V, E)$, where $m = |E|$ is the number of edges, $N \geq n$ is a known upper bound on the number of nodes $n = |V|$, and $\text{ID}_{\min}$ is the smallest ID. Moreover, the leader is the last node to terminate.*

The algorithm for Theorem 1 is presented in Algorithm 1, with the DFS notification subroutine detailed in Algorithm 2. There are five main variables used in our algorithm:

State(v): This variable represents the current state of v . We say a node v is in the *synchronized counting* phase if $\text{State}(v) = \perp$. Otherwise, the node is in the *DFS notification* phase, and $\text{State}(v) \in \{\text{Leader}, \text{Non-Leader}\}$ indicates the output of v . At the start, every node is in the synchronized counting phase. Once a node advances to the DFS notification phase, it fixes its output and cannot go back to the synchronized counting phase.

LeaderID(v): This variable stores the identifier of the leader as known by node v .

Parent(v): This variable stores the port number through which v connects to its parent in T_G .

Count(v): This variable implements the counter used in the synchronized counting phase.

$\mathcal{P}(v)$: This variable represents the remaining unexplored ports of v in the DFS notification phase. Precisely, $\mathcal{P}(v)$ is the set of ports on which v has not yet received $\text{LeaderID} + N + 2$ pulses.

For ease of presentation, we define the following two actions:

SendPulsesUntil_i(*k*): Send pulses along port *i* until a total of *k* pulses have been sent from the start of the algorithm (i.e., $\sigma_i = k$).

SendAll(1): Send one pulse along each port.

Events. For ease of analysis, several positions in Algorithm 1 and Algorithm 2 are annotated with labels: **StartDFS**(*v*) (node *v* initiates the DFS), **SendExplore_i**(*v*) (node *v* begins sending an explore-notification on port *i*), **ReceiveExplore_i**(*v*) (node *v* confirms receipt of an explore-notification from port *i*), **SendDone_i**(*v*) (node *v* begins sending a done-notification on port *i*), and **ReceiveDone_i**(*v*) (node *v* confirms receipt of a done-notification from port *i*). These labels are treated as *events* in the analysis.

Pulse labeling. Any pulse that is sent during Algorithm 2 is called a *DFS notification* pulse. Any remaining pulse is called a *synchronized counting* pulse. This distinction is used only in the analysis. The nodes cannot see these labels in the algorithm.

Algorithm 1: Leader election algorithm for node *v*

```

1 ID(v)  $\leftarrow$  ID(v) · N // Forcing all IDs to be integer multiples of N
2 State  $\leftarrow$   $\perp$ ; LeaderID  $\leftarrow$   $\perp$ ; Parent  $\leftarrow$   $\perp$ 
3 Count  $\leftarrow$  0
4 SendAll(1)
5 while State =  $\perp$  do // Synchronized counting
6   on event  $\rho_i$  is incremented for some  $i \in [\deg(v)]$  do
7     if  $\rho_i = \min_{j \in [\deg(v)]} \rho_j$  then // Counter increment condition
8       Count  $\leftarrow$  Count + 1
9       SendAll(1)
10      if Count = ID(v) then // Leader exit condition
11        State  $\leftarrow$  Leader
12        LeaderID  $\leftarrow$  ID(v)
13        Event: StartDFS(v)
14      else if  $\rho_i - \sigma_i > 1$  then // Non-Leader exit condition
15        State  $\leftarrow$  Non-Leader
16        Parent  $\leftarrow$  i
17        LeaderID  $\leftarrow$   $\lfloor \text{Count}/N \rfloor \cdot N$ 
18        wait until  $\rho_i = \text{LeaderID} + N + 2$ 
19        Event: ReceiveExplorej(v)
20    end
21 end
22 Notify(State, LeaderID, Parent) // Algorithm 2

```

Algorithm 2: Notify(State, LeaderID, Parent) for node v

```
1  $\mathcal{P} = [\deg(v)]$  // The set of all ports
2 if State = Non-Leader then
3    $\mathcal{P} \leftarrow \mathcal{P} \setminus \{\text{Parent}\}$ 
4 while  $\mathcal{P} \neq \emptyset$  do
5    $j \leftarrow \min(\mathcal{P})$  // Traversal order: prioritizing smaller port numbers
6   Event: SendExplore $_j(v)$ 
7   wait until  $\rho_j \geq \text{LeaderID} + 1$ 
8   SendPulsesUntil $_j(\text{LeaderID} + N + 2)$  // Continue DFS on port  $j$ 
9   while  $j \in \mathcal{P}$  do
10    on event  $\rho_h = \text{LeaderID} + N + 2$  for some  $h \in \mathcal{P}$  do
11      if  $j = h$  then // DFS returned from port  $j$ 
12         $\mathcal{P} \leftarrow \mathcal{P} \setminus \{j\}$ 
13        Event: ReceiveDone $_j(v)$ 
14      else // Incoming DFS from port  $h$  via an back edge
15         $\mathcal{P} \leftarrow \mathcal{P} \setminus \{h\}$ 
16        Event: ReceiveExplore $_h(v)$ 
17        Event: SendDone $_h(v)$ 
18        SendPulsesUntil $_h(\text{LeaderID} + N + 2)$ 
19    end
20  end
21 end
22 if State = Non-Leader then
23   Event: SendDone $_{\text{parent}}(v)$ 
24   SendPulsesUntil $_{\text{parent}}(\text{LeaderID} + N + 2)$  // Return DFS on parent port
```

4.1 Synchronized Counting

We begin with analyzing the synchronized counting phase of our algorithm, which corresponds to the main loop of Algorithm 1. We aim to show that the node r with the smallest identifier eventually elects itself as the leader (State = Leader) while all other nodes are still in the synchronized counting phase (State = $\perp$).

Algorithm 1 starts with Count = 0. The action SendAll(1) is performed upon initialization and immediately after each counter increment, so $\sigma_i(v) = \text{Count}(v) + 1$ for all $i \in [\deg(v)]$. The counter is incremented by one whenever an incoming pulse causes $\min_{j \in [\deg(v)]} \rho_j$ to increase, so we always have Count = $\min_{j \in [\deg(v)]} \rho_j$. There are two ways to exit the main loop of Algorithm 1. The first is Count = ID(v), which causes v to become a leader. The second is $\rho_i - \sigma_i > 1$, which causes v to become a non-leader.

Within Section 4.1, we focus on analyzing the behavior of the nodes in the synchronized counting phase. The only fact that we need to know about the DFS notification phase is that upon exiting the main loop of Algorithm 1, the value of Count is *permanently fixed*. The analysis of the DFS notification phase is done in Section 4.2.

Observation 9 (Outgoing synchronized counting pulses). *For any node $v \in V$, at any time in the execution of the algorithm, the number of synchronized counting pulses sent on each of its ports*

equals $\text{Count}(v) + 1$.

Proof. During the synchronized counting phase, the action $\text{SendAll}(1)$ is executed during initialization and immediately after each counter increment. There is no other mechanism by which node v sends synchronized counting pulses in the algorithm. Since $\text{Count}(v)$ starts at zero and becomes fixed when v exits the main loop of Algorithm 1, it follows that for any node $v \in V$, at any time during the algorithm, the number of synchronized counting pulses sent on each of its ports equals $\text{Count}(v) + 1$. $\square$

The above observation implies that for any node $v \in V$, while v is still in the synchronized counting phase, $\sigma_i(v) = \text{Count}(v) + 1$ for all $i \in [\deg(v)]$.

The following observation relates the counter value with the number of pulses sent on each port.

Observation 10 (Counter value). *For any node $v \in V$, while v is in the synchronized counting phase, $\text{Count}(v) = \min_{j \in [\deg(v)]} \rho_j(v)$.*

Proof. Since $\text{Count}(v)$ starts at zero, it suffices to show that, while v is in the synchronized counting phase, the counter is incremented by one whenever an incoming pulse causes $\min_{j \in [\deg(v)]} \rho_j$ to increase, and that there is no other mechanism for changing the counter value. This follows from the observation that a pulse arriving on port i increases $\min_{j \in [\deg(v)]} \rho_j$ if and only if $\rho_i = \min_{j \in [\deg(v)]} \rho_j$ immediately after the pulse arrives. This is precisely the condition for incrementing the counter in Algorithm 1. $\square$

Since the counter is frozen once a node exits the synchronized counting phase, there is a simple generalization of Observation 10 to any node v in any phase: $\text{Count}(v) \leq \min_{j \in [\deg(v)]} \rho_j(v)$.

The following lemma shows that the counting process is nearly synchronized along every edge, as long as it has not been disturbed by a DFS notification pulse.

Lemma 11 (Near synchronization). *For any edge $e = \{u, v\} \in E$ with no DFS notification pulse delivered yet,*

$$|\text{Count}(u) - \text{Count}(v)| \leq 1.$$

Proof. Suppose the port number of e is j at node u and i at node v . Since the counter is frozen once a node exits the synchronized counting phase, Observation 10 implies that $\text{Count}(u) \leq \rho_j(u)$. In other words, the number of pulses from v that have been delivered to u is at least $\text{Count}(u)$. Observation 9, together with the assumption that no DFS notification pulse from v has been delivered to u , implies that u receives at most $\text{Count}(v) + 1$ pulses from v , so $\rho_j(u) \leq \text{Count}(v) + 1$. Combining the two inequalities, we obtain $\text{Count}(u) \leq \text{Count}(v) + 1$. Similarly, we also have $\text{Count}(v) \leq \text{Count}(u) + 1$, so $|\text{Count}(u) - \text{Count}(v)| \leq 1$. $\square$

In the following lemma, we show that the **Non-Leader** exit condition must be triggered by a neighbor already in the DFS notification phase, meaning the first node to exit the synchronized counting phase must do so via the **Leader** exit condition.

Lemma 12 (Non-Leader exit condition). *Suppose the Non-Leader exit condition*

$$\rho_i(v) - \sigma_i(v) > 1$$

is triggered by a pulse arriving on port i at node v , then the pulse must be a DFS notification pulse.

Proof. Let u be the neighbor of v such that the port number of $e = \{u, v\}$ at node v is i . While v is in the synchronized counting phase, by Observation 9, $\sigma_i(v) = \text{Count}(v) + 1$. If v has not received any DFS notification pulse from u , then Observation 9 and Lemma 11 imply that the number of pulses v received from u is $\rho_i(v) \leq \text{Count}(u) + 1 \leq \text{Count}(v) + 2$, so

$$\rho_i(v) - \sigma_i(v) \leq 1,$$

which is insufficient to trigger the **Non-Leader** exit condition for v . $\square$

Intuitively, the following lemma shows that deadlock cannot occur during the counting process: Within a finite amount of time, the system always makes progress in at least one of the following three ways. We use the convention that $\min X = \infty$ if $X = \emptyset$. Therefore, for the special case of $S = V$, (C1) in the lemma below becomes $\min_{v \in S} \text{Count}(v) = \infty$, meaning that only (C2) or (C3) hold.

Lemma 13 (Deadlock freedom). *Let $S \subseteq V$ be the set of all nodes currently in the synchronized counting phase. Suppose no DFS notification pulses have yet been delivered to any node in S , then eventually at least one of the following conditions hold.*

(C1) $1 + \min_{u \in V \setminus S} \text{Count}(u) \leq \min_{u \in S} \text{Count}(u)$.

(C2) *Some node in S exits the synchronized counting phase.*

(C3) *A DFS notification pulse from some node in $V \setminus S$ is delivered to some node in S .*

Proof. Let $c = \min_{u \in S} \text{Count}(u)$. Select v as an arbitrary node with $\text{Count}(v) = c$. We claim that, within finite time, at least one of the following events occurs: (C1), (C2), (C3), or $\text{Count}(v) \geq c + 1$. Therefore, by applying the claim $x \cdot |S|$ times, it follows that either one of (C1), (C2), or (C3) happens, or the value of $\min_{v \in S} \text{Count}(v)$ increases by at least x . In the latter case, by choosing $x = \min_{u \in V \setminus S} \text{Count}(u)$, we ensure that (C1) happens.

For the rest of the discussion, we prove the claim. For each neighbor u of v in S , we have $\text{Count}(u) \geq c$. By Observation 9, u has already sent $\text{Count}(u) + 1 \geq c + 1$ pulses to v . If (C1) has not happened yet, for each neighbor w of v in $V \setminus S$, we also have $\text{Count}(w) \geq c$. By Observation 9, w has already sent $\text{Count}(w) + 1 \geq c + 1$ pulses to v .

Upon the arrival of these pulses, we have

$$\min_{j \in [\deg(v)]} \rho_j(v) \geq c + 1.$$

If, by this point, the **Non-Leader** exit condition $\rho_i(v) - \sigma_i(v) > 1$ for v has not been triggered yet, by Observation 10, $\text{Count}(v)$ will be incremented to $c + 1$, as required.

Now suppose the **Non-Leader** exit condition $\rho_i(v) - \sigma_i(v) > 1$ is triggered due to some incoming pulse at some point. By Lemma 12, the pulse must be a DFS notification pulse. If this pulse comes from a node in $V \setminus S$, then (C3) has happened. Otherwise, the pulse comes from a node in S , meaning that this node has already exited the synchronized counting phase, so (C2) has happened. $\square$

The following lemma shows that the counting process elects the node r with the smallest identifier as the leader, as desired.

Lemma 14 (r is the leader). *Let r be the node with the smallest identifier, then r is the first node to exit the synchronized counting phase. Moreover, r sets its state to **Leader**.*

Proof. By Lemma 12, the first node to exit the synchronized counting phase cannot do so via the **Non-Leader** exit condition. Hence it suffices to show that r is the first node that exits the synchronized counting phase.

Apply Lemma 13 with $S = V$. Since $V \setminus S = \emptyset$, (C3) is vacuously false. Moreover, since $\min_{u \in V \setminus S} \text{Count}(u) = \infty$, (C1) reduces to $\min_{v \in S} \text{Count}(v) = \infty$, which cannot hold: By the time the first node v reaches $\text{Count}(v) = \text{ID}(v)$, it must have already exited the synchronized counting phase and frozen its counter. Therefore, we conclude that (C2) must eventually occur, and hence some node v eventually exits the synchronized counting phase.

Let v be the first node to exit the synchronized counting phase. As discussed earlier, it must do so via the **Leader** exit condition, which requires $\text{Count}(v) = \text{ID}(v)$.

Suppose $v \neq r$. Consider the first moment when $\text{Count}(v) = \text{ID}(v)$. Consider any path P connecting v and r . Since no DFS notification pulse has been delivered so far, by Lemma 11, the counter values of any two adjacent nodes in P differ by at most one, so

$$\text{ID}(r) < \text{ID}(v) = \text{Count}(v) \leq \text{Count}(r) + \text{length}(P) < \text{ID}(r) + (N - 1),$$

contradicting the fact that all identifiers are integer multiples of N , so we must have $v = r$. $\square$

Remark. If we simply let the elected leader r halt after the leader election phase and do not execute Algorithm 2, we obtain a simple stabilizing leader election algorithm. By Lemma 11, for any node $v \in V \setminus \{r\}$, we have $\text{Count}(v) \leq \text{Count}(r) + \text{length}(P) < \text{ID}(r) + (N - 1) < \text{ID}(v)$, so the **Leader** exit condition cannot be triggered.

4.2 DFS Notification

We now proceed to analyze the DFS Notification phase. This phase of the algorithm employs a DFS to notify all nodes of the elected leader, while guaranteeing termination with quiescence. Although a graph can have many DFS trees, we use the *specific* tree T_G , rooted at the node with the smallest identifier and prioritizing smaller port numbers during traversal, see Section 2.3. To begin, observe that a DFS visits each edge of the graph exactly once in each direction, as detailed below.

DFS traversal. The node r with the smallest identifier initiates the DFS traversal, creates a token, and serves as the root of the DFS tree T_G . For each node v , let $\mathcal{P}(v)$ denote the set of ports from which v has not yet received the token.

When a node v holds the token, it selects the *smallest* port in $\mathcal{P}(v)$ and sends an explore-notification through that port to pass the token. If v is the first node to send the token to a neighbor u , then v is designated as the *parent* of u in the DFS tree T_G .

If $\mathcal{P}(v) = \emptyset$, it returns the token to its parent by sending a done-notification. In the special case where v is the root, this marks the completion of the entire DFS traversal.

When a node v receives an explore-notification from a neighbor u , there are two cases:

- If v previously sent the token to u and is now receiving it back, then $\{u, v\}$ is a *tree edge*, and v resumes the DFS traversal as usual.
- Otherwise, $\{u, v\}$ is a *back edge*, and v immediately returns the token to u by sending a done-notification.

A sequence of directed edges. We write

$$A = (e_1, e_2, \dots, e_{2|E|})$$

to denote the sequence of *directed edges* traversed during the DFS. For each edge $e = \{u, v\} \in E$, both directions (u, v) and (v, u) appear exactly once in A : The first occurrence corresponds to an explore-notification, and the second to a done-notification.

We now discuss how the DFS traversal is implemented in Algorithm 1 and Algorithm 2.

Sender: The moment when a node v starts sending an explore-notification or a done-notification on port j is marked as $\text{SendExplore}_j(v)$ or $\text{SendDone}_j(v)$, respectively. For an explore-notification, node v waits until $\rho_j \geq \text{LeaderID} + 1$, and then performs $\text{SendPulsesUntil}_j(\text{LeaderID} + N + 2)$. For a done-notification, node v directly performs $\text{SendPulsesUntil}_j(\text{LeaderID} + N + 2)$ without any waiting condition.

Receiver: The moment when a node v confirms the receipt of an explore-notification or a done-notification on port j is marked as $\text{ReceiveExplore}_j(v)$ and $\text{ReceiveDone}_j(v)$, respectively. The confirmation is triggered when $\rho_j = \text{LeaderID} + N + 2$.

If node v is not yet part of the DFS at the time it receives the explore-notification, there is an additional requirement: The pulses arriving on port j must first trigger the **Non-Leader** exit condition. Only after this occurs does v wait until $\rho_j = \text{LeaderID} + N + 2$ to confirm receipt of the explore-notification.

The use of $\text{SendPulsesUntil}_j(\text{LeaderID} + N + 2)$ for sending both types of notifications serves to ensure quiescence upon termination, as it allows each node to anticipate and recognize the precise number of pulses— $\text{LeaderID} + N + 2$ —it will receive before becoming inactive.

Assuming that all labeled events in the pseudocode behave as intended, the resulting execution faithfully follows the desired DFS traversal. It therefore remains to verify that the protocol functions correctly: The node with the minimum identifier is elected as the unique leader; every transmission of an explore-notification or done-notification results in confirmation of receipt of that notification; each receipt of an explore-notification or done-notification triggers the subsequent DFS action at that node; and no unintended events occur—for example, a second leader must not be elected, and synchronized counting pulses alone must not erroneously trigger confirmation of receipt for a notification that was never sent.

Notations. Consider any edge $e = \{u, v\} \in E$. Let i be the port number of e in u . Let j be the port number of e in v . For ease of presentation, we have the following notations.

$$\begin{aligned} \mathcal{E}_{u \rightarrow v}^{\text{Send}} &= \begin{cases} \text{SendExplore}_i(u) & \text{if } (u, v) \text{ is before } (v, u) \text{ in } A, \\ \text{SendDone}_i(u) & \text{otherwise.} \end{cases} \\ \mathcal{E}_{u \rightarrow v}^{\text{Receive}} &= \begin{cases} \text{ReceiveExplore}_j(v) & \text{if } (u, v) \text{ is before } (v, u) \text{ in } A, \\ \text{ReceiveDone}_j(v) & \text{otherwise.} \end{cases} \\ t_{u \rightarrow v}^{\text{Send}} &= \text{the time when the event } \mathcal{E}_{u \rightarrow v}^{\text{Send}} \text{ happens.} \\ t_{u \rightarrow v}^{\text{Receive}} &= \text{the time when the event } \mathcal{E}_{u \rightarrow v}^{\text{Receive}} \text{ happens.} \\ t_r^{\text{StartDFS}} &= \text{the time when the event } \text{StartDFS}(r) \text{ happens.} \end{aligned}$$

If an event never occurs, its occurrence time is defined as ∞ . We write

$$\mathcal{E}^* = \bigcup_{v \in V, i \in \deg(v)} \{\text{StartDFS}(v), \text{SendExplore}_i(v), \text{ReceiveExplore}_i(v), \text{SendDone}_i(v), \text{ReceiveDone}_i(v)\}$$

to denote the set of all potential events, including those that are not possible in the DFS tree T_G .

We write $e_j = (u_j, v_j)$ for each $j \in [2|E|]$. Let $R_0 = \{r\}$ and $R_i = \bigcup_{j \in [i]} \{u_j, v_j\}$ for each $i \in [2|E|]$. Intuitively, $R_i \subseteq V$ is the set of nodes that have been visited after traversing the first i directed edges $e_1, e_2, \dots, e_i$ in A .

Induction hypothesis. The analysis of the DFS notification phase is done by an induction over the indices i in the sequence $A = (e_1, e_2, \dots, e_{2|E|})$. Intuitively, the induction hypothesis for i , denoted as $\text{IH}(i)$, specifies that the DFS traversal up until the traversal of e_i works as expected, with no unintended events occurring. For any $i \in \{0, 1, \dots, 2|E|\}$, $\text{IH}(i)$ is defined as follows.

(H1) The events associated with the first i directed edges in the DFS traversal happen in order. Specifically, there exists a sequence $t_0 < t_1 < \dots < t_{i-1} < t_i$ such that

$$\begin{aligned} t_0 &= t_r^{\text{StartDFS}} = t_{u_1 \rightarrow v_1}^{\text{Send}}, \\ t_1 &= t_{u_1 \rightarrow v_1}^{\text{Receive}} = t_{u_2 \rightarrow v_2}^{\text{Send}}, \\ &\vdots \\ t_{i-1} &= t_{u_{i-1} \rightarrow v_{i-1}}^{\text{Receive}} = t_{u_i \rightarrow v_i}^{\text{Send}}, \\ t_i &= t_{u_i \rightarrow v_i}^{\text{Receive}}. \end{aligned}$$

(H2) Before time t_i , all events in

$$\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_{i-1} \rightarrow v_{i-1}}^{\text{Receive}}, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Send}} \right\}$$

do not occur.

(H3) At the time t_i , all nodes $v \in R_i$ satisfies

$$\text{ID}(r) \leq \text{Count}(v) \leq \text{ID}(r) + \text{length}(P_{v,r}) \leq \text{ID}(r) + N - 1.$$

Recall that $P_{v,r}$ is a shortest path from v to r in $\vec{G}$, see Section 2.3.

For the base case of $i = 0$, $\text{IH}(0)$ follows from Lemma 14. Specifically, for $i = 0$, (H1) asserts that $t_0 = t_r^{\text{StartDFS}} < \infty$, (H2) asserts that all events in $\mathcal{E}^* \setminus \{t_r^{\text{StartDFS}}\}$ do not occur before $t_0 = t_r^{\text{StartDFS}}$, and (H3) asserts that $\text{ID}(r) = \text{Count}(r)$ at the time $t_0 = t_r^{\text{StartDFS}}$, since $R_0 = \{r\}$.

In the following discussion, we make some observations about the induction hypothesis. For any $i \in \{0, 1, \dots, 2|E|\}$, we write $(\text{H1})(i)$, $(\text{H2})(i)$, and $(\text{H3})(i)$ to refer to the three conditions in $\text{IH}(i)$.

Observation 15 (Next sending occurs immediately). *Let $i \in \{0, 1, \dots, 2|E| - 1\}$. If $\text{IH}(i)$ holds, then the following statements hold.*

- $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$ occurs at time t_i .

- All events in $\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}, \mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} \right\}$ do not occur at time t_i .

Proof. By (H1)(i) and (H2)(i), the execution of the algorithm follows the desired DFS traversal for the first i directed edges, where the last event $\mathcal{E}$ is $\text{StartDFS}(r)$ if $i = 0$ and is $\mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}$ if $i > 0$. According to the description of our algorithm, immediately after triggering the last event $\mathcal{E}$, the current node v_{i+1} is ready to continue the DFS traversal by passing the token along the next directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$, triggering $t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$, so the first statement holds.

Other than $\mathcal{E}$ and $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$, no other event in $\mathcal{E}^*$ associated with the current node v_{i+1} is triggered at time t_i . The local computation at v_{i+1} cannot trigger any event in $\mathcal{E}^*$ associated with other nodes. Hence the second statement holds. $\square$

Observation 15 implies that, in the proof of the inductive step below, to establish (H1)($i + 1$) assuming IH(i), we may assume that the event $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$ has already occurred, and focus on proving that the corresponding event $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ eventually occurs.

The following observation formalizes the intuition that (H3) guarantees any node visited during the DFS traversal obtains correct knowledge of $\text{ID}(r)$. Consequently, in the subsequent discussion, we may replace LeaderID with $\text{ID}(r)$.

Observation 16 (Correct LeaderID). *Let $i \in \{0, 1, \dots, 2|E|\}$. If IH(i) holds, then each $v \in R_i$ has already advanced to the DFS notification phase by time t_i with $\text{LeaderID}(v) = \text{ID}(r)$.*

Proof. Suppose $v = r$. By (H1)(i), node r enters the DFS notification phase via the **Leader** exit condition at time t_0 , so $\text{LeaderID}(r) = \text{ID}(r)$. Suppose $v \neq r$. By (H1)(i), node v enters the DFS notification phase via the **Non-Leader** exit condition at some time before t_i , and sets $\text{LeaderID}(v)$ to $\lfloor \text{Count}(v)/N \rfloor \cdot N$. Since the counter value remains unchanged during the DFS notification phase, (H3)(i) implies that $\text{LeaderID}(v) = \text{ID}(r)$. $\square$

The following observation follows from the fact that the transmission of all DFS notification pulses from u to v must occur entirely within the interval $[t_{u \rightarrow v}^{\text{Send}}, t_{u \rightarrow v}^{\text{Receive}}]$. In particular, it implies that, during this interval, node u must perform the action $\text{SendPulsesUntil}_j(\text{LeaderID} + N + 2)$, where j is the port corresponding to the edge $e = \{u, v\}$ at node u , and all pulses resulting from this action are delivered to node v .

Observation 17 (DFS notification pulses). *Let $i \in \{0, 1, \dots, 2|E|\}$. If IH(i) holds, then the following holds for each edge $e = \{u, v\} \in E$ at time t_i .*

- If $(u, v) \in \{e_1, e_2, \dots, e_i\}$, then exactly $\text{ID}(r) + N + 2$ pulses have been delivered from u to v , where at least one of them must be a DFS notification pulse.
- Otherwise, u has not sent any DFS notification pulse to v .

Proof. If $(u, v) \in \{e_1, e_2, \dots, e_i\}$, then $t_{u \rightarrow v}^{\text{Receive}}$ has already occurred. From the algorithm description, this implies that v has received all the $\text{ID}(r) + N + 2$ pulses sent from u . If $(u, v) \notin \{e_1, e_2, \dots, e_i\}$, then (H2)(i) implies that u has not sent any DFS notification pulse to v .

We discuss one subtle issue in the proof: While the upper bound $\text{ID}(r) + N + 2$ appears to be clear, there might be a possibility that u already sends to v more than this number of pulses while u is in the synchronized counting phase. This is however impossible because the number of such

pulses is $\text{Count}(u) + 1 \leq (\text{ID}(r) + N - 1) + 1 < \text{ID}(r) + N + 2$ according to Observation 9 and (H3)(i). This also implies that some of the $\text{ID}(r) + N + 2$ pulses sent from u to v must be DFS notification pulses. $\square$

We now state the lemma for the inductive step.

Lemma 18. *Let $i \in \{0, 1, \dots, 2|E| - 1\}$. If $\text{IH}(i)$ holds, then $\text{IH}(i + 1)$ holds.*

Before proving Lemma 18, we first use it to prove the main theorem.

Proof of Theorem 1. The base case $\text{IH}(0)$ follows from Lemma 14, which shows that $\text{StartDFS}(r)$ occurs after finite time and is before all other events in $\mathcal{E}^*$, satisfying (H1) and (H2). The condition (H3) is satisfied because r enters the DFS notification phase through the **Leader** exit condition, which ensures $\text{Count}(r) = \text{ID}(r) = \text{ID}(r) + P_{r,r}$, as $P_{r,r} = 0$.

By Lemma 18, $\text{IH}(1), \text{IH}(2), \dots, \text{IH}(2|E|)$ hold. Given that $\text{IH}(2|E|)$ holds, we show that the algorithm terminates by time $t_{2|E|}$ with quiescence correctly. By Observation 16, all nodes v have advanced to the DFS notification phase with $\text{LeaderID}(v) = \text{ID}(r) \neq \text{ID}(v)$. This also implies that all of the nodes in $V \setminus \{r\}$ are non-leaders, so r is the unique leader.

Given that $\text{IH}(2|E|)$ holds, each edge is traversed exactly once in both directions during the DFS. For each node v , $\mathcal{E}_{u \rightarrow v}^{\text{Receive}}$ has been triggered for each of its neighbors u , meaning that $\mathcal{P}(v) = \emptyset$ and v has terminated by the time $t_{2|E|}$. Moreover, the termination comes with a quiescence guarantee by Observation 16.

The message complexity bound follows from Observation 16, which states that the number of pulses sent along each direction of each edge is exactly $\text{ID}(r) + N + 2$, where $\text{ID}(r) = \text{ID}_{\min} \cdot N$, as we multiply each node identifier by N at the beginning. Therefore, the overall message complexity is $O(m \cdot N \cdot \text{ID}_{\min})$.

The fact that the leader r is the final node to terminate follows from the ordering of events specified in (H1)(2|E|). $\square$

Inductive step. For the rest of the discussion, we prove Lemma 18. Suppose $\text{IH}(i)$ holds. Our goal is to show that $\text{IH}(i + 1)$ holds. By Observation 15, we may assume that

$$t_i = t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} < \infty$$

and take it as the starting point of the analysis. We want to show that $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ happens eventually, before triggering any other events in $\mathcal{E}^*$. The proof consists of two parts. In the first part, we show that, before $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$, no other event in $\mathcal{E}^*$ can happen, so (H2)(i + 1) holds. We also show that the following *small counter property* holds for not only the nodes in the DFS notification phase but also the remaining nodes in the synchronized counting phase. In the second part, we show that $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ occurs within finite time, so (H1)(i + 1) holds. Moreover, if this is also the moment where v_{i+1} transitions from the synchronized counting phase to the DFS notification phase, then the counter value of v_{i+1} satisfies H3(i + 1).

Definition 19 (Small counter property). *A node v is said to have the small counter property if*

$$\text{Count}(v) \leq \text{ID}(r) + \text{length}(P_{v,r}) < \text{ID}(v).$$

Yijun: checked up to here

Part 1: Bad events do not happen. We consider two bad events.

$\mathcal{E}_A$: The small counter property is violated for some node $v \in V$.

$\mathcal{E}_B$: Some event in $\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}, \mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} \right\}$ is triggered.

We aim to show that both bad events cannot happen within the time interval $\left[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}} \right)$, and moreover $\mathcal{E}_A$ does not happen at time $t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$.

Lemma 20 (Quiescence). *Suppose IH(i) holds. Consider a moment $t \geq t_i$. Suppose $\mathcal{E}_B$ has not happened, then at any moment within the time interval $[t_i, t]$, there is no DFS notification pulse in transit, except for the directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$.*

Proof. Suppose there is such a DFS notification pulse. Observe that sending such a pulse requires first triggering some event in $\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}, \mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} \right\}$, contradicting our assumption that $\mathcal{E}_B$ has not happened by time t . $\square$

Lemma 21 (Chain and anchor). *Suppose IH(i) holds. Consider a moment $t \geq t_i$. Suppose no DFS notification pulse has been delivered to any node in $V \setminus R_i$, except that there might be DFS notification pulse delivered along the directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$, then all nodes $v \in V$ satisfy the small counter property.*

Proof. If $v \in R_i$, then the small counter property follows from (H3)(i). Now consider any $v \in V \setminus R_i$. We use the chain-and-anchor argument. Consider the first node w in the shortest path $P_{v,r}$ that belongs to R_i , so

$$\text{length}(P_{v,r}) = \text{length}(P_{v,w}) + \text{length}(P_{w,r}).$$

By (H3)(i), $\text{Count}(w) \leq \text{ID}(r) + \text{length}(P_{w,r})$. By Lemma 11, $\text{Count}(v)$ is at most

$$\text{Count}(w) + \text{length}(P_{v,w}) \leq \text{ID}(r) + \text{length}(P_{w,r}) + \text{length}(P_{v,w}) = \text{ID}(r) + \text{length}(P_{v,r}),$$

as required.

For the calculation to work, we need to ensure that no DFS notification pulse has been delivered along any edge in $P_{v,w}$ in any direction. This requires that $P_{v,w}$ does not traverse along the edge $\{u_{i+1}, v_{i+1}\}$. As $u_{i+1} \in R_i$, such a traversal is possible only when $v_{i+1} \notin R_i$, in which case the edge is oriented as (u_{i+1}, v_{i+1}) in $\vec{G}$, so it is impossible that $P_{v,w}$ contains (u_{i+1}, v_{i+1}) , since otherwise we would have selected $w = u_{i+1}$. $\square$

Lemma 22. *Suppose IH(i) holds. Consider a moment $t \geq t_i$. If both $\mathcal{E}_A$ and $\mathcal{E}_B$ have not happened and the following statements hold.*

- *The next pulse delivery also does not trigger $\mathcal{E}_A$.*
- *If the next pulse delivery does not trigger $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$, then it also does not trigger $\mathcal{E}_B$.*

Proof. By Lemma 20, the next pulse delivered—as well as any pulse delivered after time t_i and before time t —is either a DFS notification pulse along the directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$ or a synchronized counting pulse. Combining this with Observation 17, the precondition of Lemma 21 is satisfied at the moment after the next pulse delivery, so Lemma 21 implies that the next pulse delivery does not trigger $\mathcal{E}_A$.

Suppose the next pulse delivery triggers an event $\mathcal{E}$ in

$$\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}, \mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} \right\}.$$

Let v be the node that receives this pulse.

Since the pulse delivery does not trigger $\mathcal{E}_A$, we know that the counter value at any node v after the pulse delivery is at most $\text{ID}(r) + N - 1$, so $\mathcal{E}$ cannot be $\text{StartDFS}(v)$, as $\text{ID}(r) + N - 1 < \text{ID}(v)$ for any $v \in V \setminus \{r\}$.

If the pulse is a DFS notification pulse along the directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$, then $\mathcal{E}$ must be $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$. Therefore, for the rest of the discussion, we assume that the pulse is a synchronized counting pulse. In particular, Lemma 12 implies that the pulse cannot trigger the **Non-Leader** exit condition.

Therefore, the only remaining way to trigger any event in $\mathcal{E}^*$ is to get a total of $\text{ID}(r) + N + 2$ pulses via some port, but that is not possible with synchronized counting pulses only, unless the small counter property is violated. Specifically, Observation 9 implies that the number of synchronized counting pulses sent along a port equals the counter value plus one. While the small counter property holds, this number is capped at $(\text{ID}(r) + N - 1) + 1$, which is smaller than the required $\text{ID}(r) + N + 2$.

Therefore, we conclude that, if the next pulse delivery does not trigger $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$, then it does not trigger $\mathcal{E}_B$. $\square$

Lemma 23 (Bad events do not happen). *Event $\mathcal{E}_A$ does not happen within the time interval $[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}]$, and event $\mathcal{E}_B$ does not happen within the time interval $[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}]$.*

Proof. We prove the lemma by an induction over the number of pulses sent after time t_i .

For the base case of the induction, consider the moment $t = t_i$. By Observation 15, $\mathcal{E}_B$ has not happened yet. By a combination of Observation 17 and Lemma 20, the precondition of Lemma 21 is satisfied at $t = t_i$, so $\mathcal{E}_A$ has not happened yet.

For the inductive step, apply Lemma 22 to any moment $t \in [t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}]$. Suppose, by induction hypothesis that both $\mathcal{E}_A$ and $\mathcal{E}_B$ have not happened by time t , then it is guaranteed that the next pulse delivery does not trigger $\mathcal{E}_A$. Moreover, if the next pulse delivery does not trigger $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$, then it also does not trigger $\mathcal{E}_B$. Therefore, $\mathcal{E}_A$ does not occur in the time interval $[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}]$, and $\mathcal{E}_B$ does not occur in the time interval $[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}]$. $\square$

Part 2: The next event happens. After getting rid of the bad events $\mathcal{E}_A$ and $\mathcal{E}_B$, the next thing to do is to show that $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ happens within finite time.

Lemma 24 (The next event happens). *If $\text{IH}(i)$ holds, then $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ occurs within finite time. Moreover, $\text{Count}(v_{i+1}) \geq \text{ID}(r)$ at time $t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$.*

Proof. Let the port of the edge $\{u_{i+1}, v_{i+1}\}$ be j in u_{i+1} and k in v_{i+1} . By Observation 15, $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$ occurs at time t_i . We break the analysis into a few cases.

Done-notification. Suppose a done-notification is sent across e_{i+1} , meaning that $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$ is $\text{SendDone}_j(v_{i+1})$ and $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ is $\text{ReceiveDone}_k(v_{i+1})$. The way our algorithm sends a done-notification implies that a total of $\text{ID}(r) + N + 2$ pulses have been sent from u_{i+1} along the edge $\{u_{i+1}, v_{i+1}\}$ by time t_i . To confirm the receipt of the done-notification, it suffices that all these $\text{ID}(r) + N + 2$ pulses are delivered v_{i+1} , which happens within a finite time.

Explore-notification. Suppose an explore-notification is sent across e_{i+1} . That is, $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$ is $\text{SendExplore}_j(v_{i+1})$ and $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ is $\text{ReceiveExplore}_k(v_{i+1})$. By the description of our algorithm, upon triggering the event $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}}$, u_{i+1} first waits until getting at least $\text{ID}(r) + 1$ messages from v_{i+1} . After that, u_{i+1} sends to v_{i+1} several pulses until a total of $\text{ID}(r) + N + 2$ pulses have been sent along the directed edge $e_{i+1} = (u_{i+1}, v_{i+1})$.

Back edge. Suppose $\{u_{i+1}, v_{i+1}\}$ is a back edge, meaning that v_{i+1} has already advanced to the DFS notification phase by time t_i . By (H3)(i), $\text{Count}(v_{i+1}) \geq \text{ID}(r)$, so v_{i+1} has already sent to u_{i+1} at least $\text{Count}(v_{i+1}) + 1 \geq \text{ID}(r) + 1$ synchronized counting pulses (Observation 9), meaning that u_{i+1} only waits for a finite amount of time, and eventually v_{i+1} will receive a total of $\text{ID}(r) + N + 2$ pulses from u_{i+1} , triggering $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$.

Tree edge. Suppose $\{u_{i+1}, v_{i+1}\}$ is a tree edge, meaning that v_{i+1} is still in the synchronized counting phase by time t_i .

First, we claim that the waiting before sending pulses takes finite time for u_{i+1} by applying Lemma 13 to the moment t_i with $S = V \setminus R_i$. At least one of the three types of progress is made within finite time.

- Suppose (C1) holds, then

$$1 + \text{ID}(r) \leq 1 + \min_{u \in V \setminus S} \text{Count}(u) \leq \min_{v \in S} \text{Count}(v) \leq \text{Count}(v_{i+1}),$$

where the first inequality follows from (H3)(i). By Observation 9, this means that v_{i+1} has sent out $\text{ID}(r) + 1$ pulses to u_{i+1} , and eventually these pulses will arrive at u_{i+1} , so u_{i+1} only waits for a finite time.

- Suppose (C2) holds, meaning that some node in S exits the synchronized counting phase, so either $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ is triggered, or some event in

$$\mathcal{E}^* \setminus \left\{ \text{StartDFS}(r), \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Send}}, \mathcal{E}_{u_1 \rightarrow v_1}^{\text{Receive}}, \dots, \mathcal{E}_{u_i \rightarrow v_i}^{\text{Receive}}, \mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} \right\}$$

is triggered. In the former case, we are done already. The latter case is not possible before first triggering $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ in view of Lemma 23.

- Suppose (C3) holds, meaning that a DFS notification pulse from some node in $V \setminus S$ is delivered to some node in S . By a combination of Lemma 20 and Lemma 23, we infer that unless $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ has been triggered, the pulse must be sent from u_{i+1} to v_{i+1} . In either case, the waiting must be done within finite time.

After the waiting is done, eventually v_{i+1} will receive a total of $\text{ID}(r) + N + 2$ pulses from u_{i+1} , meaning that

$$\rho_k(v_{i+1}) = \text{ID}(r) + N + 2.$$

For v_{i+1} to confirm the receipt of the explore-notification, we need v_{i+1} to first satisfy the **Non-Leader** exit condition. To do so, we use the small counter property, which is guaranteed to hold during the time interval $\left[t_i, t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}\right)$ in view of Lemma 23.

Due to the small counter property, the counter value of v_{i+1} is capped by $\text{ID}(r) + N - 1$, so by Observation 9, while v_{i+1} is still in the synchronized counting phase,

$$\sigma_k(v_{i+1}) \leq (\text{ID}(r) + N - 1) + 1 = \text{ID}(r) + N.$$

Therefore, there must be a moment when the **Non-Leader** exit condition $\rho_k - \sigma_k > 1$ is satisfied for v_{i+1} , and then once all the $\text{ID}(r) + N + 2$ pulses from u_{i+1} have been delivered to v_{i+1} , $\mathcal{E}_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}} = \text{ReceiveExplore}_k(v_{i+1})$ is triggered. $\square$

Part 3: Wrapping up. Now we have collected all the needed ingredients to prove Lemma 18.

Proof of Lemma 18. Let $i \in \{0, 1, \dots, 2|E| - 1\}$. Suppose $\text{IH}(i)$ holds. Our goal is to show that $\text{IH}(i + 1)$ holds. To prove $(\text{H1})(i + 1)$, it suffices to show that

$$t_i = t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} < t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}} < \infty.$$

The part $t_i = t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Send}} < t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$ follows from Observation 15. The part $t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}} < \infty$ follows from Lemma 24. We set $t_{i+1} = t_{u_{i+1} \rightarrow v_{i+1}}^{\text{Receive}}$. To prove $(\text{H2})(i + 1)$, it suffices to show that, within the timer interval $[t_i, t_{i+1})$, all events in $\mathcal{E}_B$ do not occur. This follows from Lemma 23. To prove $(\text{H3})(i + 1)$, it suffices to show that

$$\text{ID}(r) \leq \text{Count}(v_{i+1}) \leq \text{ID}(r) + \text{length}(P_{v_{i+1}, r})$$

at time t_{i+1} . If we already have $v_{i+1} \in R_i$, then this follows from $(\text{H3})(i)$. Otherwise, by Lemma 23 again, we infer that the small counter property holds for v_{i+1} at time t_{i+1} , so $\text{Count}(v_{i+1}) \leq \text{ID}(r) + \text{length}(P_{v_{i+1}, r})$. The inequality $\text{ID}(r) \leq \text{Count}(v_{i+1})$ follows from Lemma 24. $\square$

5 Conclusions and Open Questions

In this paper, we demonstrate that quiescently terminating content-oblivious leader election is achievable in general 2-edge-connected networks. Previously, such leader election was only known to be possible in ring topologies—specifically, a quiescently terminating algorithm in oriented rings and a stabilizing algorithm in unoriented rings [FGG⁺24]. Consequently, we remove the need for a preselected leader in the general algorithm simulation result by Censor-Hillel, Cohen, Gelles, and Sela [CCG⁺23].

Several intriguing open questions remain. The foremost is whether a *uniform* content-oblivious leader election algorithm—one that requires no prior knowledge of the network—is possible in general 2-edge-connected networks. So far, this has only been shown for ring topologies [FGG⁺24]. Our current algorithm requires knowledge of an upper bound N on the number of nodes n . Specifically, node identifiers must be spaced at least N apart to prevent any node other than the leader r

from satisfying the leader termination condition $\text{Count}(v) = \text{ID}(v)$. This dependence on N appears inherent to our counting-based approach, where counters of adjacent nodes may differ by one.

Another important open question is whether the efficiency of our algorithm can be improved. Due to the use of DFS, our algorithm is highly sequential. A common way to measure *time complexity* of an asynchronous algorithm is to assume that each message takes one unit of time to be delivered [AW04]. Under this model, the time complexity of our algorithm is $O(m \cdot N \cdot \text{ID}_{\min})$, matching its message complexity, as the DFS traverses the m edges sequentially, and the traversal of each edge requires sending $O(N \cdot \text{ID}_{\min})$ pulses. It remains an open question whether we can reduce this time complexity by making the algorithm less sequential.

Much less is known about lower bounds for content-oblivious leader election. The only known result is a lower bound of $\Omega(n \cdot \log \frac{k}{n})$, where k is the number of distinct identifiers in the network, due to Frei, Gelles, Ghazy, and Nolin [FGG⁺24]. Currently, there remains a significant gap between this lower bound and the upper bounds, both in rings and in general 2-edge-connected networks. A key open question is whether the message complexity for general 2-edge-connected networks is inherently higher than that of ring topologies. If so, how can we leverage the structural properties of the hard instances to establish such a lower bound?

References

- [AAD⁺06] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, 2006.
- [ARV22] Dan Alistarh, Joel Rybicki, and Sasha Voitovich. Near-optimal leader election in population protocols on graphs. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 246–256, 2022.
- [AW04] Hagit Attiya and Jennifer Welch. *Distributed computing: fundamentals, simulations, and advanced topics*. John Wiley & Sons, 2004.
- [Awe85] Baruch Awerbuch. Complexity of network synchronization. *Journal of the ACM (JACM)*, 32(4):804–823, 1985.
- [BDM93] Michael Barborak, Anton Dahbura, and Mirosław Malek. The consensus problem in fault-tolerant computing. *ACM Computing Surveys (CSur)*, 25(2):171–220, 1993.
- [BGK20] Petra Berenbrink, George Giakkoupis, and Peter Kling. Optimal time and space leader election in population protocols. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 119–129, 2020.
- [CCG⁺23] Keren Censor-Hillel, Shir Cohen, Ran Gelles, and Gal Sela. Distributed computations in fully-defective networks. *Distributed Computing*, 36(4):501–528, 2023.
- [CD19] Artur Czumaj and Peter Davies. Leader election in multi-hop radio networks. *Theor. Comput. Sci.*, 792(C):2–11, November 2019. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2019.02.027. URL: <https://doi.org/10.1016/j.tcs.2019.02.027>.
- [CD21] Artur Czumaj and Peter Davies. Exploiting spontaneous transmissions for broadcasting and leader election in radio networks. *Journal of the ACM (JACM)*, 68(2):1–22, 2021.
- [CGH19] Keren Censor-Hillel, Ran Gelles, and Bernhard Haeupler. Making asynchronous distributed computations robust to noise. *Distributed Computing*, 32:405–421, 2019.

- [CK03] Imrich Chlamtac and Shay Kutten. On broadcasting in radio networks-problem analysis and protocol design. *IEEE Transactions on Communications*, 33(12):1240–1246, 2003.
- [CK10] Alejandro Cornejo and Fabian Kuhn. Deploying wireless networks with beeps. In *Proceedings of the 24th International Conference on Distributed Computing*, DISC’10, pages 148–162, Cambridge, MA. Springer-Verlag, 2010. ISBN: 3642157629.
- [CKP⁺19] Yi-Jun Chang, Tsvi Kopelowitz, Seth Pettie, Ruosong Wang, and Wei Zhan. Exponential separations in the energy complexity of leader election. *ACM Transactions on Algorithms*, 15(4), 2019. DOI: 10.1145/3341111.
- [DBB18] Fabien Dufoulon, Janna Burman, and Joffroy Beauquier. Beeping a deterministic time-optimal leader election. In *32nd International Symposium on Distributed Computing (DISC 2018)*, pages 20–1. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018.
- [Dol82] Danny Dolev. The byzantine generals strike again. *Journal of algorithms*, 3(1):14–30, 1982.
- [DS18] David Doty and David Soloveichik. Stable leader election in population protocols requires linear time. *Distributed Computing*, 31(4):257–271, 2018.
- [Dub13] Elena Dubrova. *Fault-Tolerant Design*. Springer, Berlin, 2013. DOI: 10.1007/978-1-4614-2113-9.
- [EK21] Yuval Emek and Eyal Keren. A thin self-stabilizing asynchronous unison algorithm with applications to fault tolerant biological networks. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 93–102, 2021.
- [EW13] Yuval Emek and Roger Wattenhofer. Stone age distributed computing. In *Proceedings of the 2013 ACM symposium on Principles of distributed computing (PODC)*, pages 137–146, 2013.
- [FGG⁺24] Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. Content-Oblivious Leader Election on Rings. In Dan Alistarh, editor, *38th International Symposium on Distributed Computing (DISC 2024)*, volume 319 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 26:1–26:20, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. ISBN: 978-3-95977-352-2. DOI: 10.4230/LIPIcs.DISC.2024.26. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2024.26>.
- [FLP85] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)*, 32(2):374–382, 1985.
- [FP23] Orr Fischer and Merav Parter. Distributed CONGEST algorithms against mobile adversaries. In Rotem Oshman, Alexandre Nolin, Magnús M. Halldórsson, and Alkida Balliu, editors, *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 262–273. ACM, 2023. DOI: 10.1145/3583668.3594578. URL: <https://doi.org/10.1145/3583668.3594578>.
- [FSW14] Klaus-Tycho Förster, Jochen Seidel, and Roger Wattenhofer. Deterministic leader election in multi-hop beeping networks. In *Distributed Computing: 28th International Symposium, DISC 2014, Austin, TX, USA, October 12-15, 2014. Proceedings 28*, pages 212–226. Springer, 2014.
- [Gel17] Ran Gelles. Coding for interactive communication: a survey. *Foundations and Trends® in Theoretical Computer Science*, 13(1–2):1–157, 2017.

- [GH13] Mohsen Ghaffari and Bernhard Haeupler. Near optimal leader election in multi-hop radio networks. In *Proceedings of the twenty-fourth annual ACM-SIAM symposium on Discrete algorithms*, pages 748–766. SIAM, 2013.
- [HP21a] Yael Hitron and Merav Parter. Broadcast CONGEST algorithms against adversarial edges. In Seth Gilbert, editor, *Proceedings of the 35th International Symposium on Distributed Computing (DISC)*, volume 209 of *LIPIcs*, 23:1–23:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. DOI: 10.4230/LIPIcs.DISC.2021.23. URL: <https://doi.org/10.4230/LIPIcs.DISC.2021.23>.
- [HP21b] Yael Hitron and Merav Parter. General CONGEST compilers against adversarial edges. In Seth Gilbert, editor, *Proceedings of the 35th International Symposium on Distributed Computing (DISC)*, volume 209 of *LIPIcs*, 24:1–24:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. DOI: 10.4230/LIPIcs.DISC.2021.24. URL: <https://doi.org/10.4230/LIPIcs.DISC.2021.24>.
- [HPY22] Yael Hitron, Merav Parter, and Eylon Yogev. Broadcast CONGEST algorithms against eavesdroppers. In Christian Scheideler, editor, *Proceedings of the 36th International Symposium on Distributed Computing (DISC)*, volume 246 of *LIPIcs*, 27:1–27:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. URL: <https://doi.org/10.4230/LIPIcs.DISC.2022.27>.
- [HPY23] Yael Hitron, Merav Parter, and Eylon Yogev. Secure distributed network optimization against eavesdroppers. In Yael Tauman Kalai, editor, *Proceedings of the 14th Innovations in Theoretical Computer Science Conference (ITCS)*, volume 251 of *LIPIcs*, 71:1–71:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. DOI: 10.4230/LIPIcs.ITCS.2023.71. URL: <https://doi.org/10.4230/LIPIcs.ITCS.2023.71>.
- [JKL15] Abhishek Jain, Yael Tauman Kalai, and Allison Bishop Lewko. Interactive coding for multiparty protocols. In *Proceedings of the 2015 Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 1–10, 2015.
- [KK20] Israel Koren and C. Mani Krishna, editors. *Fault-Tolerant Systems*. Morgan Kaufmann, San Francisco, 2nd edition, 2020. DOI: 10.1016/B978-0-12-818105-8.
- [Par22] Merav Parter. A graph theoretic approach for resilient distributed algorithms. In Alessia Milani and Philipp Woelfel, editors, *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, page 324. ACM, 2022. DOI: 10.1145/3519270.3538453. URL: <https://doi.org/10.1145/3519270.3538453>.
- [Pel92] Andrzej Pelc. Reliable communication in networks with byzantine link failures. *Networks*, 22(5):441–459, 1992.
- [PY19a] Merav Parter and Eylon Yogev. Distributed algorithms made secure: A graph theoretic approach. In Timothy M. Chan, editor, *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1693–1710. SIAM, 2019. DOI: 10.1137/1.9781611975482.102. URL: <https://doi.org/10.1137/1.9781611975482.102>.
- [PY19b] Merav Parter and Eylon Yogev. Secure distributed computing made (nearly) optimal. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 107–116. ACM, 2019. DOI: 10.1145/3293611.3331620. URL: <https://doi.org/10.1145/3293611.3331620>.

- [Ray18] Michel Raynal. *Fault-Tolerant Message-Passing Distributed Systems: An Algorithmic Approach*. Springer, Cham, 2018. DOI: 10.1007/978-3-319-94141-7.
- [Rob39] Herbert Ellis Robbins. A theorem on graphs, with an application to a problem of traffic control. *The American Mathematical Monthly*, 46(5):281–283, 1939.
- [RS94] Sridhar Rajagopalan and Leonard Schulman. A coding theorem for distributed computation. In *Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pages 790–799, 1994.
- [Sch92] Leonard J Schulman. Communication on noisy channels: a coding theorem for computation. In *Proceedings., 33rd Annual Symposium on Foundations of Computer Science*, pages 724–733. IEEE Computer Society, 1992.
- [Sch93] Leonard J Schulman. Deterministic coding for interactive communication. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 747–756, 1993.
- [Sch96] Leonard J Schulman. Coding for interactive communication. *IEEE Transactions on Information Theory*, 42(6):1745–1756, 1996. DOI: 10.1109/18.556671.
- [SM20] Yuichi Sudo and Toshimitsu Masuzawa. Leader election requires logarithmic time in population protocols. *Parallel Processing Letters*, 30(01):2050005, 2020.
- [VZ25] Robin Vacus and Isabella Ziccardi. Minimal leader election under weak communication, 2025. arXiv: 2502.12697 [cs.DC]. URL: <https://arxiv.org/abs/2502.12697>.